Cairo University

Faculty of Engineering

Department of Electronics and Electrical Communications

# Modulation Project

Prepared by


| ID | Section | BN. | Name |
|---|---|---|---|
| **9230862** | 4 | 18 | مريم جمعة خلف مصطفى |
| **9230937** | 4 | 24 | ميرنا ناجي فرح حنا سلامة |
| **9230653** | 3 | 23 | فاطمه محمود فولي علام |
| **9230591** | 3 | 10 | علياء محمود السيد إبراهيم |


Course: **Digital Communications**

Under the supervision of: **Eng. Saleh Ramadan**

# Table of Contents

# Table of figures

# 1. Introduction

This project involves simulating a baseband single carrier communication system to evaluate BPSK, QPSK, 8PSK, BFSK, and 16QAM modulation schemes. By passing data through an AWGN channel and calculating the Bit Error Rate (BER) , the objective is to compare simulated performance against theoretical bounds across various $E_b/N_0$ levels

# 2. BPSK Analysis

## 2.1 BPSK system Overview

Binary Phase Shift Keying (BPSK) is a digital modulation technique that transmits data by shifting the phase of a carrier signal between two possible states. It is one of the simplest and most robust modulation schemes, offering strong noise immunity compared to higher-order modulation techniques.

In BPSK, each symbol represents one bit, mapped to one of two possible phase states (typically 0° and 180°). Although it is less bandwidth-efficient than QPSK, it provides better performance in noisy environments due to the larger distance between constellation points.

## 2.2 System Implementation

- **Mapper:**

In the BPSK transmitter, a random binary stream is generated, where each bit is treated as a separate symbol since each symbol carries only one bit. Unlike QPSK, no grouping or decimal conversion is required.

Each bit is directly mapped to a real-valued symbol. Typically, binary 0 is mapped to -1, and binary 1 is mapped to +1. These two values represent the two-phase states of the BPSK signal. The mapped symbols lie on the real axis, forming a one-dimensional constellation with two points.

This simple mapping process produces the transmitted BPSK symbols, which are then passed through the communication channel.

**Matlab Code**:

```
%% Parameters
N = 1e5;
bits = randi([0 1], N, 1);

%% MAPPER
symbols = 2*bits - 1;
```

- **Channel:**

The transmitted BPSK symbols are passed through an Additive White Gaussian Noise (AWGN) channel to simulate real communication conditions. A range of $E_b/N_0$ values from −4 dB to 14 dB is used to evaluate system performance under different noise levels, from low SNR (high noise) to high SNR (low noise).

For each $E_b/N_0$ value, it is first converted to linear scale to compute the noise power spectral density $N_0$. The noise variance is then determined based on the bit energy $E_b$. Since BPSK is a real-valued modulation scheme, real Gaussian noise is generated using randn and scaled by $\sqrt{N_0/2}$.
This generated noise is added to the transmitted symbols, producing the received noisy symbols that will later be processed by a simple threshold-based demapper to recover the transmitted bits.

## Matlab Code:

```
%% Channel

EbNo_dB = -4:1:14;
BER = zeros(1, length(EbNo_dB));
Eb = 1;

for i = 1:length(EbNo_dB)

    EbNo_linear = 10^(EbNo_dB(i)/10);
    N0 = Eb / EbNo_linear;

    noise = sqrt(N0/2) * randn(size(symbols));
    rx_symbols = symbols + noise;

    %% DEMAPPER
    rx_bits = rx_symbols > 0;

    %% BER
    BER(i) = sum(bits ~= rx_bits) / N;

End
```

- **Demapper:**

At the receiver, the noisy BPSK symbols are detected using a simple threshold decision rule instead of Euclidean distance calculation. Since BPSK uses a one-dimensional constellation with two possible values (+1 and −1), each received symbol is compared to a zero threshold. Symbols greater than zero are decided as binary 1, while symbols less than zero are decided as binary 0.
The detected bits are then directly reconstructed without any need for decimal conversion or bit grouping.
The Bit Error Rate (BER) is computed by comparing the received bits with the transmitted bits and dividing the number of errors by the total number of bits $N$. This process is repeated for all $E_b/N_0$ values to evaluate system performance under different noise levels.

## Matlab Code:

```
%% DEMAPPER
    rx_bits = rx_symbols > 0;
```

---

```
%% Theoretical BER
EbNo_linear = 10.^(EbNo_dB/10);
BER_theoretical = 0.5 * erfc(sqrt(EbNo_linear));
```

## 2.3 Simulation Results

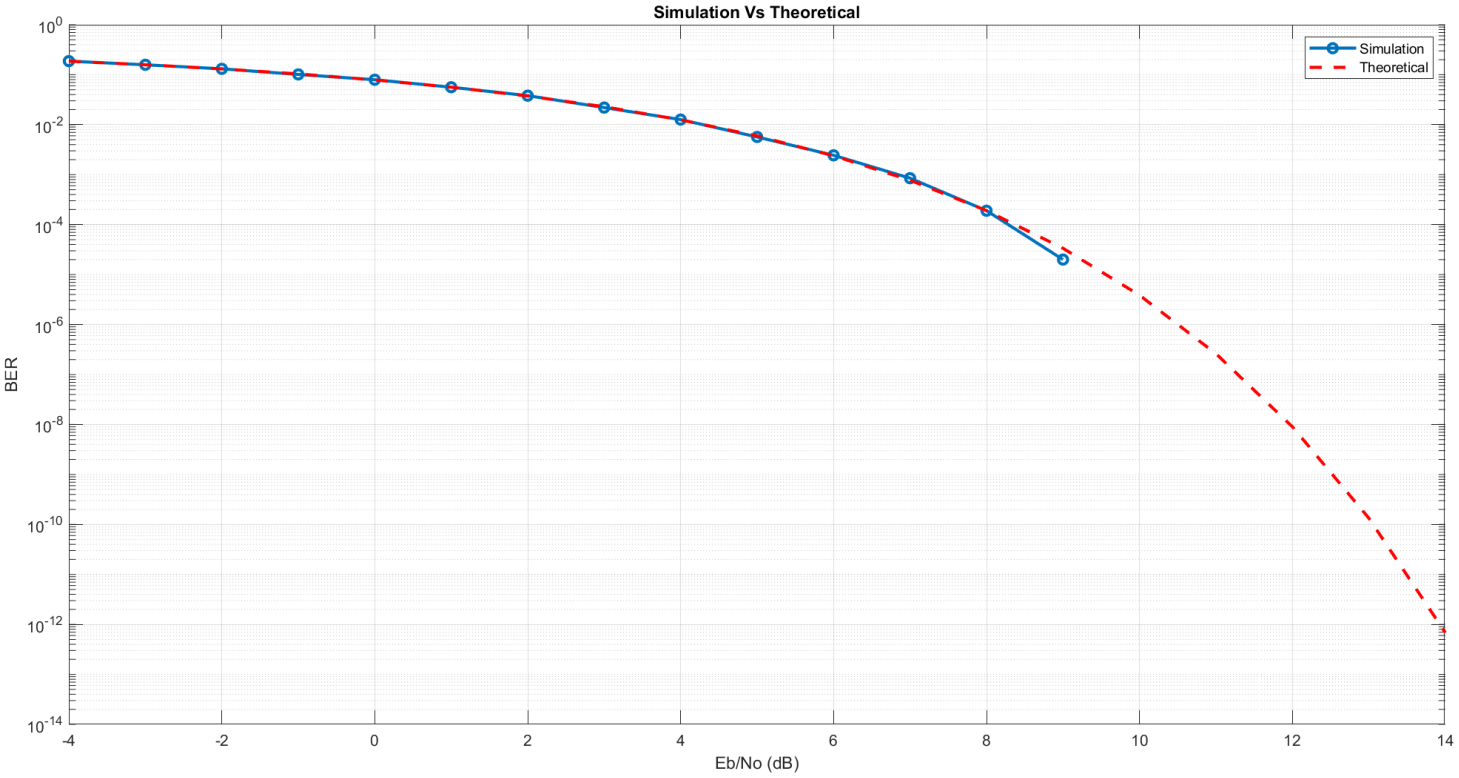



**Figure 1 BPSK – Simulated vs. Theoretical BER**

## Comment:

The results show a strong agreement between the simulated BER and the theoretical BER for both Gray and non-Gray coding schemes, confirming that the simulation is correctly implemented and follows the expected analytical behavior. At moderate $E_b/N_0$values, the simulated curves closely match the theoretical ones.

However, at higher $E_b/N_0$values, a slight deviation is observed. This is mainly due to the finite number of transmitted bits used in the simulation, which limits the probability of observing errors in high SNR regions. As a result, the simulated BER may drop to zero when no errors are detected. Since the plot uses a logarithmic scale, a BER value of zero cannot be displayed (as $\log(0)$is undefined), causing the simulated curve to stop or terminate earlier. In contrast, the theoretical BER, derived from the Q-function, decreases smoothly and asymptotically without reaching zero.

# 3. QPSK Analysis

## 3.1 QPSK system Overview

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique that transmits data by varying the phase of a carrier signal. It is widely used because it offers better bandwidth efficiency than binary modulation schemes.

In QPSK, each symbol represents two bits, mapped to one of four possible phase states. This allows the data rate to be doubled compared to BPSK without increasing the required bandwidth.

## 3.2 System Implementation

- **Mapper:**

In the QPSK transmitter, a random binary stream is generated and grouped into pairs since each symbol carries two bits. Each pair is converted into a decimal value (0–3) to simplify constellation indexing.
Two mappings are used: Gray mapping, where adjacent symbols differ by only one bit to reduce errors, and Non-Gray mapping, where this property is not guaranteed. Each decimal value is then mapped to its corresponding complex symbol in the I-Q plane For example, the decimal value 0 is mapped to the symbol -1-j, which corresponds to the first constellation point., forming the transmitted QPSK symbols for both cases.

### Matlab Code:

```
%% Parameters
N = 1e5; % Number of bits
bits = randi([0 1], N, 1);
% Group bits into pairs [b1, b2]
bit_pairs = reshape(bits, 2, [])';
%% MAPPER
% Convert binary pairs to decimal (0, 1, 2, 3)
dec_values = bi2de(bit_pairs, 'left-msb');
% Define Constellations (Decimal indices: 0, 1, 2, 3)
gray_constellation    = [-1-1j; -1+1j;  1-1j;  1+1j];
nongray_constellation = [-1-1j; -1+1j;  1+1j;  1-1j];

% Map the symbols
symbols_gray    = gray_constellation(dec_values + 1);
symbols_nongray = nongray_constellation(dec_values + 1);
```

- **Channel:**

The transmitted QPSK symbols are passed through an Additive White Gaussian Noise (AWGN) channel to simulate real communication conditions. A range of $E_b/N_0$ values from −4 dB to 14 dB is used to evaluate system performance under different noise levels, from low SNR (high noise) to high SNR (low noise).
For each $E_b/N_0$ value, it is first converted to linear scale to compute the noise power spectral density $N_0$. The noise variance is then determined based on the bit energy $E_b$. Complex Gaussian noise is generated using randn for both real and imaginary components and scaled by $\sqrt{N_0/2}$.

This generated noise is added to the transmitted symbols, producing the received noisy symbols that will later be processed by the demapper.

## Matlab Code:

```
%% AWGN CHANNEL
EbNo_dB = -4:1:14; % Range of Eb/No in decibels
BER_gray = zeros(1, length(EbNo_dB));
BER_nongray = zeros(1, length(EbNo_dB));

Es = 2; % Energy per symbol (1^2 + 1^2 = 2)
Eb = Es / 2; % Energy per bit

for i = 1:length(EbNo_dB)

   % Calculate Noise Variance based on Eb/No
   EbNo_linear = 10^(EbNo_dB(i)/10);
   N0 = Eb / EbNo_linear;

   % Generation of Complex AWGN using randn
   noise_gray = sqrt(N0 / 2) * (randn(size(symbols_gray)) +1j*randn(size(symbols_gray)));
        noise_nongray = sqrt(N0 / 2) * (randn(size(symbols_nongray))+ 1j*randn(size(symbols_nongray)));

   % received symbols with noise
   rx_symbols_gray = symbols_gray + noise_gray;
   rx_symbols_nongray = symbols_nongray + noise_nongray;
```

- ## Demapper:

At the receiver, each noisy symbol is compared with all constellation points by computing the Euclidean distance. The closest point (minimum distance) is selected as the detected symbol. Its index is then converted to the corresponding decimal value (subtracting 1 due to MATLAB indexing), and then back to the original two-bit representation.

The Bit Error Rate (BER) is computed by comparing the received bits with the transmitted bits and dividing the number of errors by the total number of bits $N$. This process is repeated for all $E_b/N_0$values to evaluate system performance under different noise levels.

## Matlab Code:

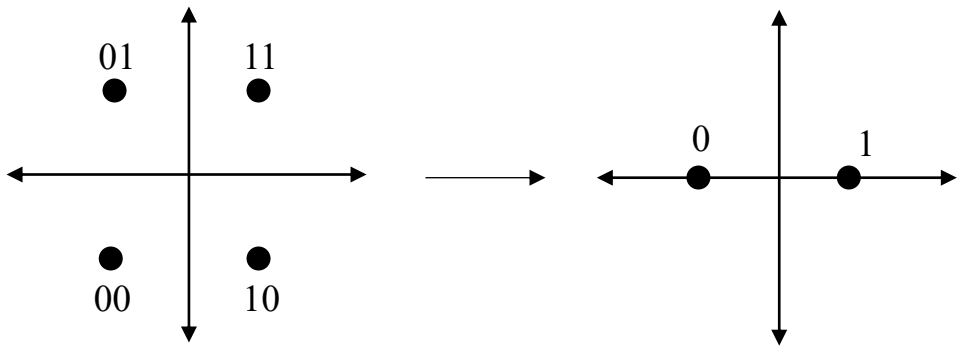
```
%% DEMAPPER
   rx_bits_gray = zeros(size(bits));
   rx_bits_nongray = zeros(size(bits));

   for k = 1:length(rx_symbols_gray)
      % Gray Demapping
      distances_gray = abs(rx_symbols_gray(k) - gray_constellation);
      [value1 , min_index_gray] = min(distances_gray);
      dec_val_gray = min_index_gray - 1;
      rx_bits_gray(2*k-1:2*k) = de2bi(dec_val_gray, 2, 'left-msb')';

      % Non-Gray Demapping
      distances_nongray = abs(rx_symbols_nongray(k) - nongray_constellation);
      [value2 , min_index_nongray] = min(distances_nongray);
```

```
    dec_val_nongray = min_index_nongray - 1;
    rx_bits_nongray(2*k-1:2*k) = de2bi(dec_val_nongray, 2, 'left-msb')';
end

%% BER CALCULATOR
num_errors_gray    = sum(bits ~= rx_bits_gray);
num_errors_nongray = sum(bits ~= rx_bits_nongray);

% Store the Bit Error Rate for Eb/No value
BER_gray(i) = num_errors_gray / N;
BER_nongray(i) = num_errors_nongray / N;
```

## 3.3 Theoretical Analysis

- **QPSK with Gray Mapping:**

In Gray-coded QPSK, the in-phase (I) and quadrature (Q) components can be treated as two independent BPSK systems. The first bit is determined by the sign of the real part of the received symbol (left corresponds to 0 and right to 1), while the second bit is determined by the sign of the imaginary part. Due to Gray mapping, adjacent symbols differ by only one bit, so most symbol errors result in a single bit error.



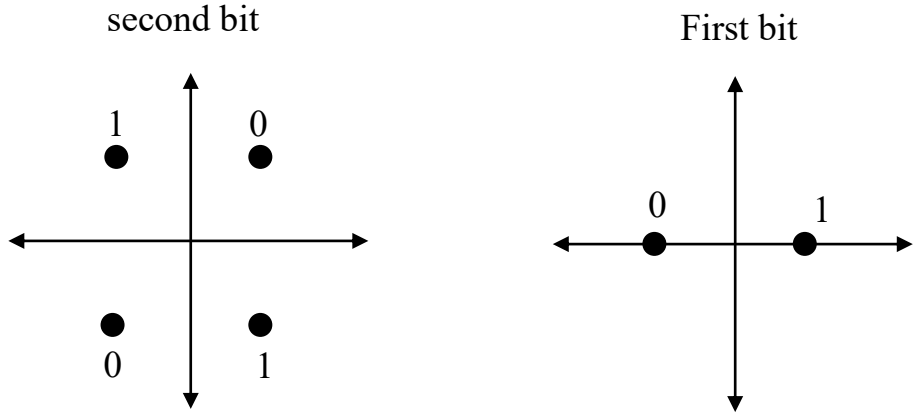

$$\therefore P_{error\ 1^{st}\ bit} = P_{error\ 2^{nd}\ bit} = \frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right)$$

$$\therefore BER = \frac{P_{error\ 1^{st}\ bit} + P_{error\ 2^{nd}\ bit}}{2} = \mathbf{\frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right)}$$

- **QPSK with Non-Gray Mapping**

In non-Gray QPSK, the first bit is determined by the sign of the real part (left = 0, right = 1), similar to Gray mapping. However, the second bit depends on the quadrant: the first and third quadrants correspond to one value, while the second and fourth correspond to the other. This arrangement means adjacent symbols may differ by more than one bit, increasing the chance of multiple bit errors.

To find the error probability of the second bit, the conditional error probabilities for all four symbols are averaged. Due to symmetry, opposite quadrants have equal probabilities, and the average is obtained by summing over all cases and dividing by 4. As a result, the BER is higher than in Gray mapping.




$$P_{error}(0 \mid S_2) = P_{error}(0 \mid S_0) = \frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right) + \frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right)$$

$$= erfc\left(\sqrt{\frac{E_b}{N_0}}\right) \ \text{"two neighbours"}$$

$$P_{error}(1 \mid S_1) = P_{error}(1 \mid S_3) = \frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right) + \frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right) = erfc\left(\sqrt{\frac{E_b}{N_0}}\right)$$

$$\therefore P_{error\ 2^{nd}\ bit} = \frac{P_{error}(0 \mid S_2) + P_{error}(0 \mid S_0) + P_{error}(1 \mid S_1) + P_{error}(1 \mid S_3)}{4}$$

$$= erfc\left(\sqrt{\frac{E_b}{N_0}}\right)$$

$$\therefore BER = \frac{P_{error\ 1^{st}\ bit} + P_{error\ 2^{nd}\ bit}}{2} = \frac{\frac{1}{2} erfc\left(\sqrt{\frac{E_b}{N_0}}\right) + erfc\left(\sqrt{\frac{E_b}{N_0}}\right)}{\mathbf{2}} = \mathbf{0.75}\ \boldsymbol{erfc}\left(\sqrt{\frac{\boldsymbol{E_b}}{\boldsymbol{N_0}}}\right)$$

## 3.4 Simulation Results

- Gray QPSK – Simulated vs. Theoretical BER

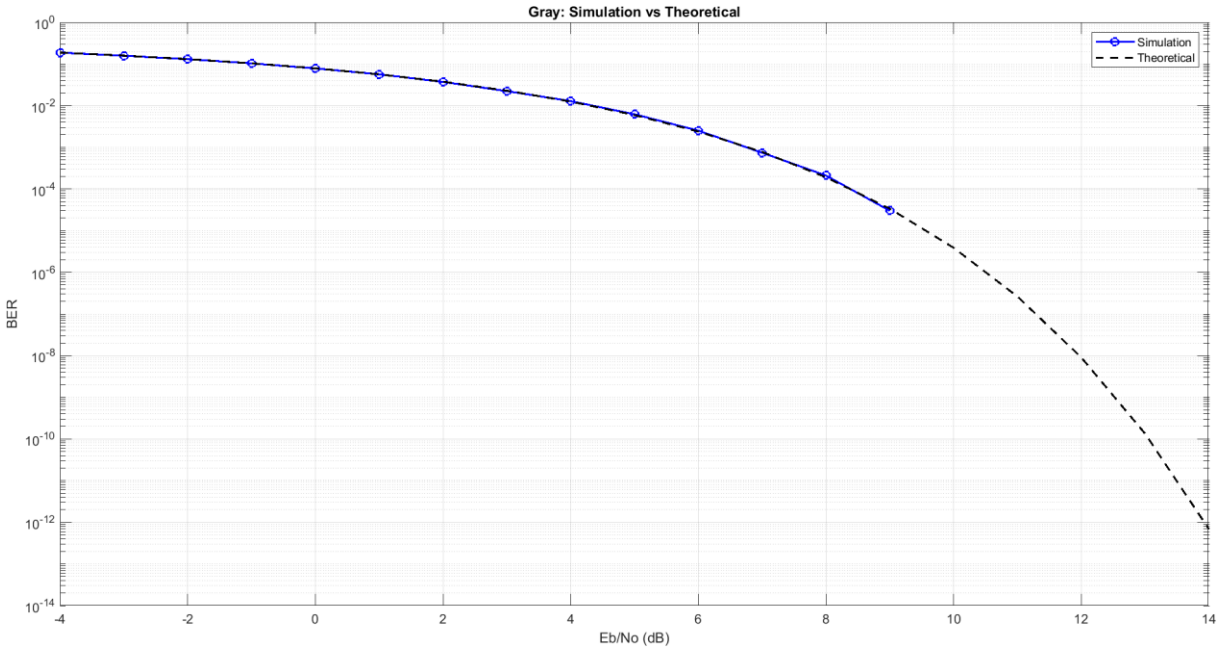



**Figure 2 Gray QPSK – Simulated vs. Theoretical BER**

- Non-Gray QPSK – Simulated vs. Theoretical BER

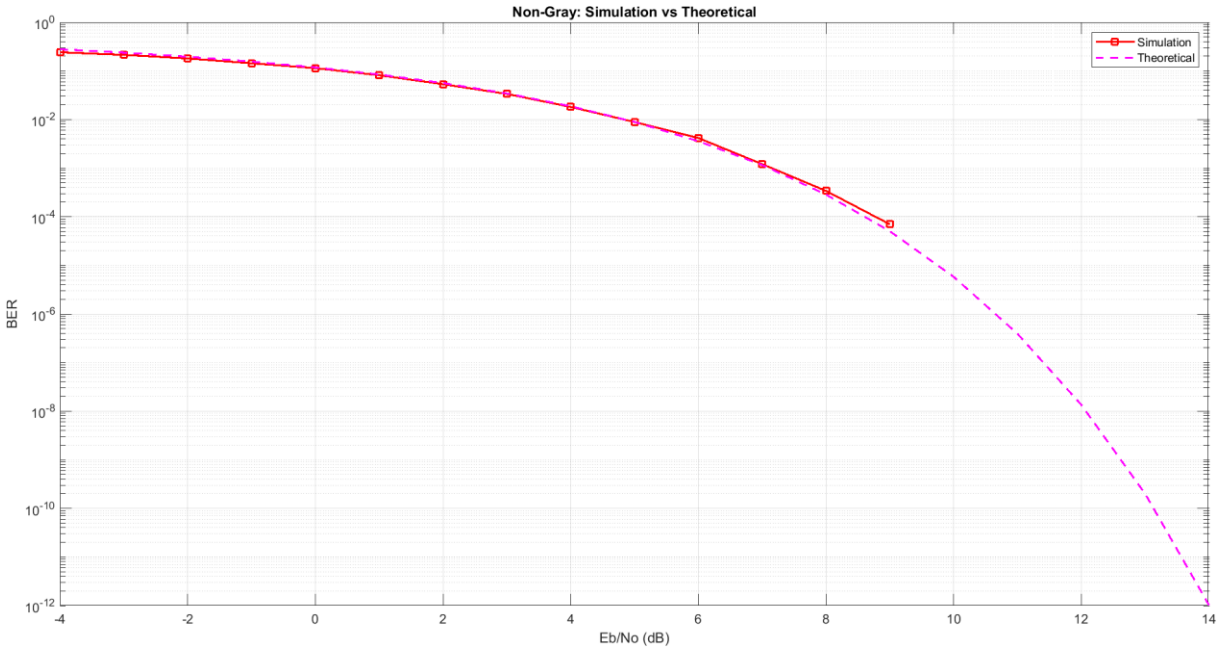



**Figure 3 Non-Gray QPSK – Simulated vs. Theoretical BER**

**Comment:**

The results show a strong agreement between the simulated BER and the theoretical BER for both Gray and non-Gray coding schemes, confirming that the simulation is correctly implemented

and follows the expected analytical behavior. At moderate SNR (Eb/No) values, the simulated curves closely match the theoretical ones. However, at higher SNR values, a slight deviation appears. This occurs because the theoretical BER is based on the error function (erfc), which decreases smoothly and never actually reaches zero. In contrast, the simulated BER is obtained by counting errors, so when no errors are detected at high SNR values (e.g. around 9–10 dB), the BER becomes zero. Since the plot is on a logarithmic scale, log(0) is undefined (−∞), causing the simulated curve to stop or not extend further, while the theoretical curve continues smoothly.

- Gray vs Non-Gray (Simulated + Theoretical)

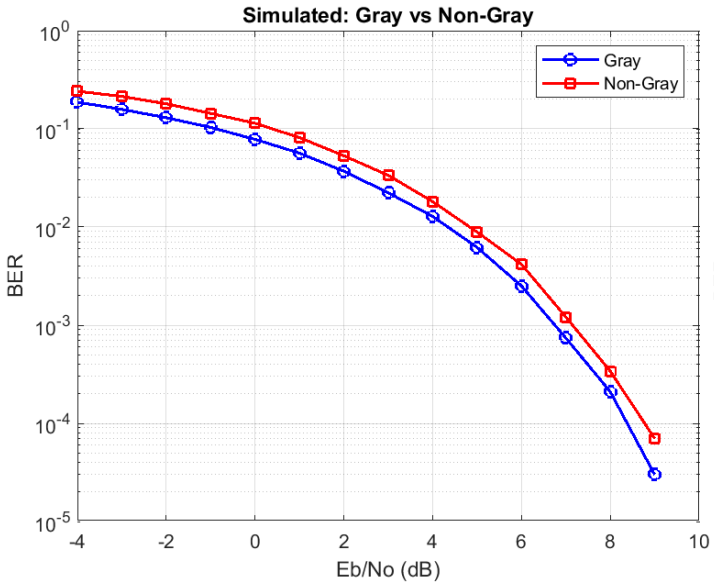



**Figure 4 Simulated BER Comparison (Gray vs. Non-Gray)**

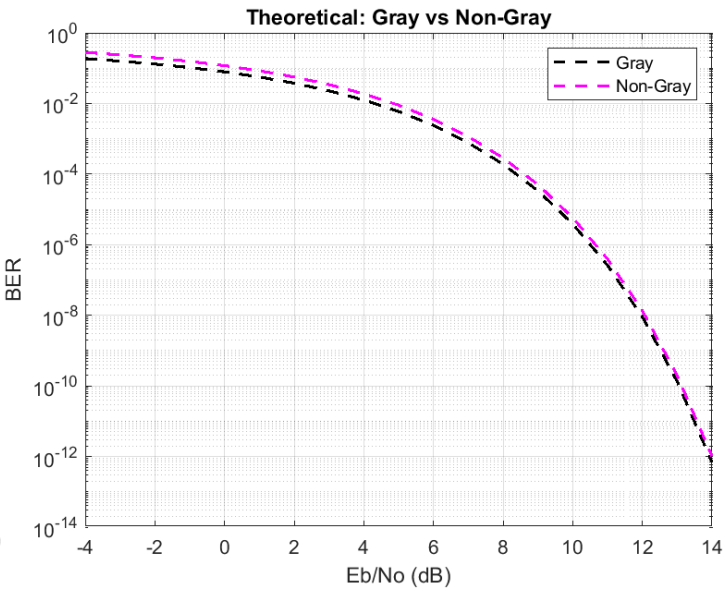



**Figure 5 Theoretical BER Comparison (Gray vs. Non-Gray)**

## Comment:

Both the theoretical and simulated results clearly show that non-Gray coding has a higher BER than Gray coding at all SNR values. This is because Gray coding minimizes the number of bit errors per symbol error, as only one bit changes between adjacent symbols. In contrast, non-Gray coding may cause multiple bit errors from a single symbol error, which increases the overall BER. Additionally, this difference can be approximated theoretically, where the BER of non-Gray coding is about 1.5 times higher than that of Gray coding, as indicated by $0.75\ \mathrm{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right)$ compared to $0.5\ \mathrm{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right)$. Therefore, both the analytical and simulation results consistently confirm that Gray coding provides better error performance than non-Gray coding.

# 4. 8PSK Analysis

## 4.1 8PSK System Overview

8-Phase Shift Keying (8PSK) is a digital modulation technique in which information is transmitted by varying the phase of a constant-amplitude carrier signal. Each transmitted symbol carries 3 bits (since $\log_2 8 = 3$ ), meaning 8PSK achieves a spectral efficiency of 3 bits per symbol. The constellation consists of 8 equally spaced points on a unit circle, separated by a phase difference of 2π/8 radians (45°) between adjacent symbols. And Gray coding is applied so that adjacent symbols differ by only one bit, which minimises the bit-error rate at high SNR.

In comparison with BPSK (1 bit/symbol) and QPSK (2 bits/symbol), 8PSK offers higher spectral efficiency at the cost of higher noise sensitivity, since the Euclidean distance between adjacent constellation points decreases as the number of phases increases.

## 4.2 System Implementation

- **Mapper:**

In the 8PSK transmitter, a random binary stream is generated and grouped into sets of three bits since each symbol carries three bits of information. Each group is converted into a decimal value (0–7).
Gray mapping to symbols is used, where adjacent symbols differ by only one bit to reduce the probability of multiple bit errors during symbol detection.
Each decimal value is then mapped to a corresponding complex symbol located on the unit circle in the
I-Q plane. The symbols are defined as:

$$S_i(t) = \left(\sqrt{\frac{2E}{T}}\, cos(2\pi f_c t + (i-1)\frac{2\pi}{M}\right) \quad 0 \le t < T, i = 1,2,\ldots\ldots,M, such\ as\ M = 8$$

**MATLAB Code**:

```
%% Parameters
order = 8;                  % 8-PSK
no_of_bits = log2(order);   % 3 bits per symbol
symorder = 'gray';          % Gray mapping
%% Constellation Generation
symbols = 0:order-1;        % create symbol indices [0 1 2 3 4 5 6 7]
psk_symbols = pskmod(symbols, order, 0, symorder);      % generate 8-PSK constellation points, and apply gray mapping
n_symbols = length(psk_symbols);
gray_bits = de2bi(symbols, no_of_bits, 'left-msb');     % convert each symbol index into binary
```

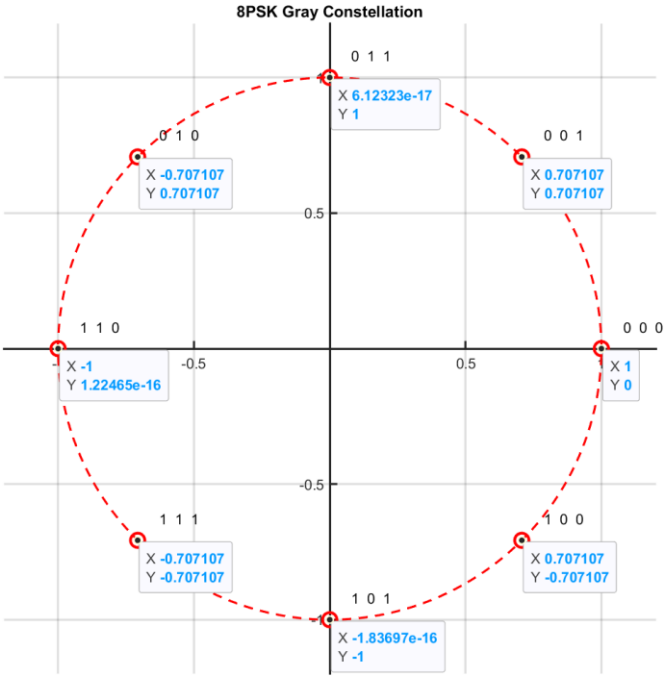



**Figure 6 Constellation of 8PSK**

- ## Channel:

The transmitted 8PSK symbols are passed through an Additive White Gaussian Noise (AWGN) channel to simulate real-world communication conditions. A range of $E_b/N_0$ values from −4 dB to 14 dB is used to evaluate system performance under different noise levels.
For each $E_b/N_0$, it is converted from dB to linear scale to compute the noise power spectral density $N_0$. The noise variance is derived based on the bit energy $E_b$.
Complex Gaussian noise is generated using independent Gaussian random variables for the in-phase and quadrature components, scaled by: $\sqrt{N_0/2}$ This noise is added to the transmitted symbols to produce the received noisy signal.

## MATLAB Code:

```
%% BER Parameters
EbNo_dB = -4:1:14;      % SNR range in dB
BER = zeros(size(EbNo_dB));
Es = 1;     % Symbol energy normalized to 1
Eb = Es / no_of_bits;

%% Channel Simulation
for i = 1:length(EbNo_dB)
   EbNo_linear = 10^(EbNo_dB(i)/10);      % convert dB to linear scale
   N0 = Eb / EbNo_linear;                 % Noise power spectral density
   % Generate random transmitted symbols
   Nsym = 1e5;
   tx_index = randi([0 7], Nsym, 1);
   % Map indices to PSK symbols
   tx_symbols = psk_symbols(tx_index + 1);
   % Convert symbols to bit stream vector
   tx_bits = de2bi(tx_index, no_of_bits, 'left-msb');
   tx_bits = reshape(tx_bits.', [], 1);
   % Generates complex Gaussian noise
```

```
noise = sqrt(N0/2) * (randn(size(tx_symbols)) + 1j*randn(size(tx_symbols)));
rx_symbols = tx_symbols + noise;    % Received signal is equal to transmitted + noise
```

- **Demapper:**

At the receiver, The demapper applies a minimum Euclidean distance decision rule. For each received complex sample. The symbol with the minimum distance is selected as the detected symbol (Maximum Likelihood Detection).
Once the detected symbol index is determined, it is de-mapped back to its Gray-coded bit sequence. The resulting bit stream is compared with the transmitted bit stream to count the number of bit errors, from which the BER is computed over all $E_b/N_0$ values.

**MATLAB Code**:

```
%% Demapper
rx_index = zeros(Nsym,1);
for n = 1:Nsym
   [~, idx] = min(abs(rx_symbols(n) - psk_symbols));   % Find nearest constellation point
   rx_index(n) = idx - 1;
end
% Convert detected symbols to bit stream
rx_bits = de2bi(rx_index, no_of_bits, 'left-msb');
rx_bits = reshape(rx_bits.', [], 1);
%% BER
BER(i) = sum(tx_bits ~= rx_bits) / length(tx_bits);
end
```

## 3.3 Theoretical Analysis

For M-PSK with Gray coding, the exact symbol error rate (SER) over an AWGN channel is given by the well-known result. For 8PSK (M = 8), the SER can be bounded or approximated using the union bound. For Gray-coded M-PSK the approximate bit error probability is:

$$SER \leq \frac{M-1}{2 * \log_2 \mathrm{M}} erfc\left(\sqrt{\frac{E_s}{N_0} sin(\frac{\pi}{M})}\right) \quad , \mathrm{M} = 8 \text{ for 8PSK}$$

But we will use the tight bound formula from hand analysis

- **Hand Analysis**

$$\boldsymbol{P(e \mid s_i) = P(error\ to\ left\ neighbor) + P(error\ to\ right\ neighbor)}$$

$$\boldsymbol{P(e \mid s_i)} = \frac{1}{2} erfc\left(\sqrt{\frac{E_s}{N_0} sin\left(\frac{\pi}{8}\right)}\right) + \frac{1}{2} erfc\left(\sqrt{\frac{E_s}{N_0} sin\left(\frac{\pi}{8}\right)}\right)$$

$$= erfc\left(\sqrt{\frac{E_s}{N_0} sin\left(\frac{\pi}{8}\right)}\right)$$

**As All symbols are equally likely:**

$$\boldsymbol{P_s = SER} = erfc\left(\sqrt{\frac{E_s}{N_0}sin\left(\frac{\pi}{8}\right)}\right)$$

$$\textbf{As } BER = \frac{SER}{\log_2 8}, \qquad E_s = E_b \log_2 8 \qquad , for\ 8PSK$$

$$\boldsymbol{BER} = \frac{1}{3}erfc\left(\sqrt{\frac{3E_b}{N_0}sin\left(\frac{\pi}{8}\right)}\right)$$

**Note :**The factor sin(π/8) ≈ 0.3827 reflects the minimum angular separation between adjacent 8PSK symbols. This factor is smaller than the equivalent for QPSK (sin(π/4) ≈ 0.7071), which confirms that 8PSK requires a higher $E_b/N_0$ to achieve the same BER as QPSK.

## 3.4 Simulation Results

The simulated BER was obtained by transmitting $N_s = 10^5$ symbols per SNR point over the$E_b/N_0$ range $\sqrt{4}$dB to 14 dB. **Figure 6** compares the simulation results with the theoretical curve derived before.

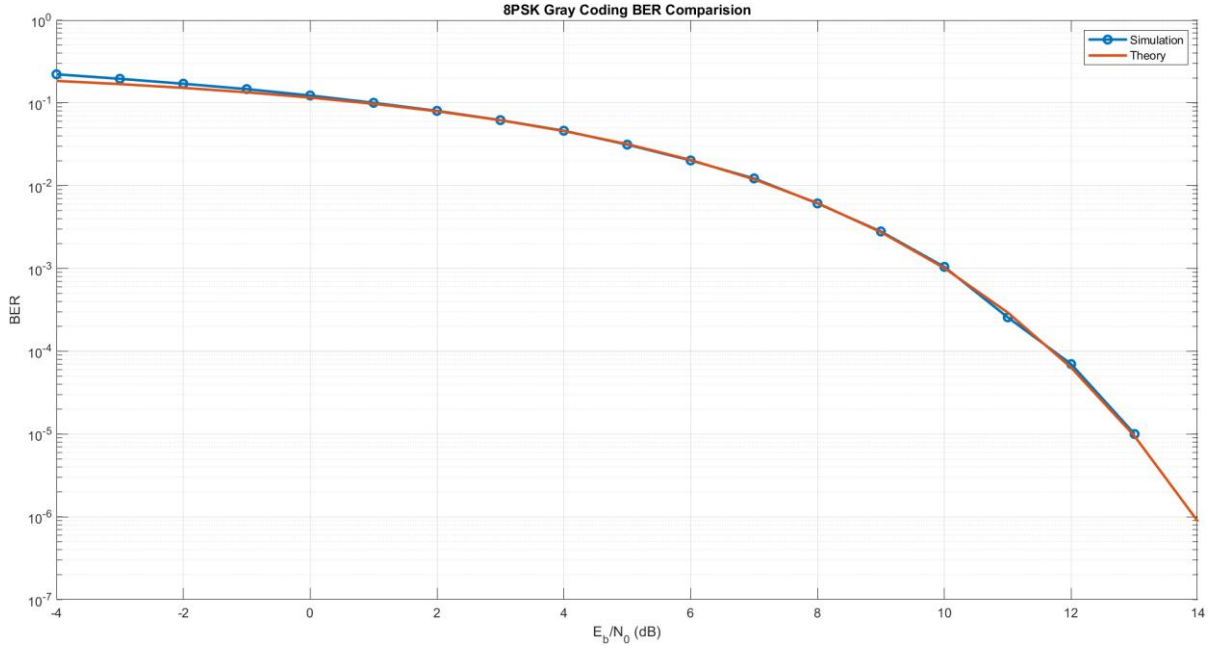



**Figure 7 Gray 8PSK – Simulated vs. Theoretical BER**

## Comment:

The results show that the simulated BER closely follows the theoretical curve. At low and moderate Eb/No values(From −4 dB up to approximately 10 dB), the agreement is very strong. The slight upward deviation of the simulation at very low SNR (−4 dB to −2 dB) is expected because the higher variance when error counts are large. At higher Eb/No values(≥ 11 dB), the simulated BER deviates slightly at that begining due to the finite number of transmitted bits, where zero errors may occur, causing the curve to drop.

To achieve a BER of $e.i\ 10^{-3}$, 8PSK requires approximately Eb/No =10 dB. Compared with QPSK, which achieves the same BER at around 6.5 dB, 8PSK incurs a penalty of roughly 3.5 dB. However, 8PSK achieves higher spectral efficiency by transmitting 3 bits per symbol instead of 2.

# 5. 16 QAM Analysis

## 5.1 16_QAM system Overview

Quadrature Amplitude modulation (QAM) is a digital modulation technique that transmits data by varying the phase and amplitude of a carrier signal.

Formula $s_i(t) = \sqrt{\frac{2E_0}{T}} a_i \cos(2\pi f_c t) - \sqrt{\frac{2E_0}{T}} bi \sin(2\pi f_c t)$ , $where\ a, b = \pm 1, \pm 3$

So, we have two main basis function sine, cosine so the transmitted signal can be represented as

$X_{PB} = X_I + X_\varphi$ where I represent the in-phase component (cos component) and Q quadrature component (sine component).

In 16-QAM each symbol represents four bits, mapped to one of 16 possible symbols. This allows higher data rates.

## 5.2 System Implementation

- **Mapper:**

First, we generate many random binary bits consist of (10000 *16) symbols as we increase the number of bits its behavior becomes close to theoretical 16-QAM, each symbol carries 4 bits because $\log_2(16) = 4$.

After generating the binary data, each group of 4 bits is converted into a decimal value using binary-to-decimal conversion with MSB-first representation. This step transforms every 4-bit sequence into an integer value ranging from 0 to 15

At the end each decimal value is mapped to a corresponding complex symbol using a 16-QAM table. This table contains the 16 possible constellation points formed by combinations of in-phase (I) and quadrature (Q) amplitude levels as gray coding to minimize the error.

**Matlab Code**:

```
%% generate and mapping the data to send (MAPPER)
number_of_symbols=10000*16;

number_of_bits=4;

        data_bits =randi([0 1],number_of_symbols,number_of_bits); %each row represent a symbol 4 bits
data_decimal= bi2de(data_bits,'left-msb');
%each symbole is consist of I and Q component
QAM16_table =[-3-3j;-3-j;-3+3j;-3+j;
        -1-3j;-1-j;-1+3j;-1+j;
```

```
3-3j;3-j;3+3j;3+j;
1-3j;1-j;1+3j;1+j];

%mapping
 symboles=QAM16_table(data_decimal+1);%add one because matlab indexing start from 1 not     zero
```

- ## Channel:

In a 16-QAM communication system, the channel is modeled as an Additive White Gaussian Noise (AWGN) channel which is $\sim N\left(0, \frac{N_0}{2}\right)$. The signal-to-noise ratio (SNR) is defined in dB and varied from -4 dB to 14 dB to evaluate system performance under different channel conditions.

First, the SNR in dB is converted into a linear scale using $10^{\frac{SNR}{10}}$ . The noise power spectral density $N_0$is then calculated using the relationship between bit energy $E_b$and SNR, where $N_0 = \frac{E_b}{SNR_{linear}}$. And the Eb is calculated from $E_b = \frac{E_{savg}}{log_2 16}$ as the energy of each symbol not equal in 16_QAM so the $E_{savg}$ is equal to$= \frac{\Sigma E^2}{16} = \frac{4(\sqrt{2})^2+8(\sqrt{10})^2+4(\sqrt{18})^2}{16} = 10$ so the Eb theoretically =2.5.

Finally, the channel noise is generated as complex Gaussian noise using the MATLAB **randn** function for both real and imaginary components. Each component is scaled by $\sqrt{N_0/2}$. This noise is then added to the transmitted 16-QAM symbols, resulting in a noisy received signal that simulates real wireless channel conditions. This received signal is later used for demodulation and BER calculation.

## Matlab Code:

```
SNR=-4:14; %in db Eb/NO

demapped_bits=zeros(size(data_bits));

theoritical_BER = zeros(length(SNR),1);

Esav=sum((abs(symboles)).^2)/number_of_symbols ;%calculate the Esavrage to get the noise poer for symbol not bit

Eb=Esav/4;

for i=1:length (SNR)
SNR_linear=10^(SNR(i)/10);

No=Eb/SNR_linear;

noise = sqrt(No/2)*(randn(size(symboles))+1j*randn(size(symboles)));

noisy_signal=noise+symboles;
```

- ## Demapper:

At first we divide the received 16-QAM signal into its in-phase (I) and quadrature (Q) components using the real and imaginary parts of the complex signal. Then the decision regions are defined using threshold values at -2, 0, and 2, which correspond to the midpoints between the

constellation levels (-3, -1, +1, +3) as shown in fig4. Based on these thresholds, each received sample is classified into one of four regions, and each region is mapped back to a 2-bit binary value. In the demapping process, a loop is used to process each received symbol individually. For every symbol, the I component is first compared against the decision boundaries to determine its corresponding 2-bit sequence.

The same process is applied to the Q component. The two resulting 2-bit sequences are then concatenated to reconstruct the original 4-bit transmitted symbol. This produces the estimated transmitted bit stream, stored in the demapped_bits matrix.

To calculate the BER we calculate the number of bit errors using a mismatch operation between the transmitted bits and received bit after demapping process, then dividing the total number of errors by the total number of transmitted bits. This process is repeated for each SNR value. In addition, the theoretical BER is computed using an analytical expression for 16-QAM in AWGN channels.

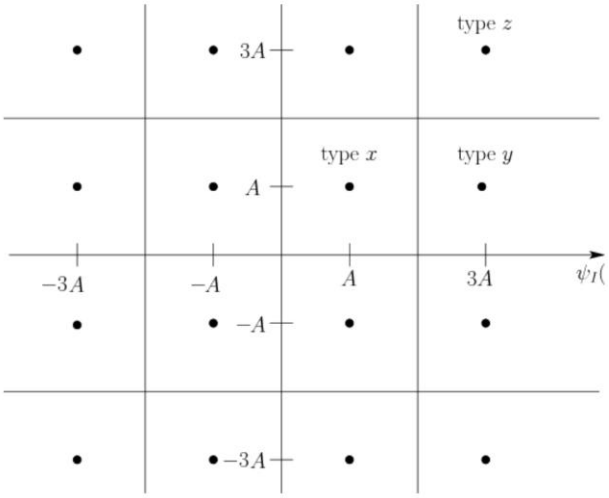



**Figure 8 the decision region for 16_QAM**

## Matlab Code:

```
%% demapper using desicion regions of 16QAM
recieved_signal=noisy_signal;
XI=real(recieved_signal);
XQ=imag(recieved_signal);
for j=1:number_of_symbols
      if (XI(j)<-2)
     XI_bit=[0 0];
      elseif (XI(j)<0)
     XI_bit=[0 1];
      elseif (XI(j)<2)
     XI_bit=[1 1];
      else
     XI_bit=[1 0];
      end
     if (XQ(j)<-2)
     XQ_bit=[0 0];
     elseif (XQ(j)<0)
     XQ_bit=[0 1];
     elseif (XQ(j)<2)
```

```
        XQ_bit=[1 1];
        else
        XQ_bit=[1 0];
        end
demapped_bits(j,:) = [XI_bit XQ_bit] ;
end
%% BER
number_of_errors=sum(data_bits ~=demapped_bits);
BER_simulated (i,:) =number_of_errors/(number_of_symbols) ;
BER_avg =(BER_simulated (:,1)+BER_simulated (:,2)+BER_simulated (:,3)+BER_simulated (:,4))*0.25;
theoritical_BER(i,:)=(3/8)*erfc(sqrt(Eb/(2.5*No)));
```

## 5.3 Theoretical Analysis

- **16_QAM with Gray Mapping:**

In Gray-coded 16-QAM, the in-phase (I) and quadrature (Q) components can be treated as four independent ASK systems, Each of the I and Q components behaves like a 4-level amplitude shift keying (4-ASK) system, where two bits are mapped to one of four amplitude levels. Therefore, instead of directly analyzing the full 16-QAM constellation, the problem can be simplified by studying the probability of error for a 4-ASK system. This significantly reduces the complexity of the analysis, since the 1-dimensional case is much easier to handle than the full 2-dimensional constellation as shown in fig5.

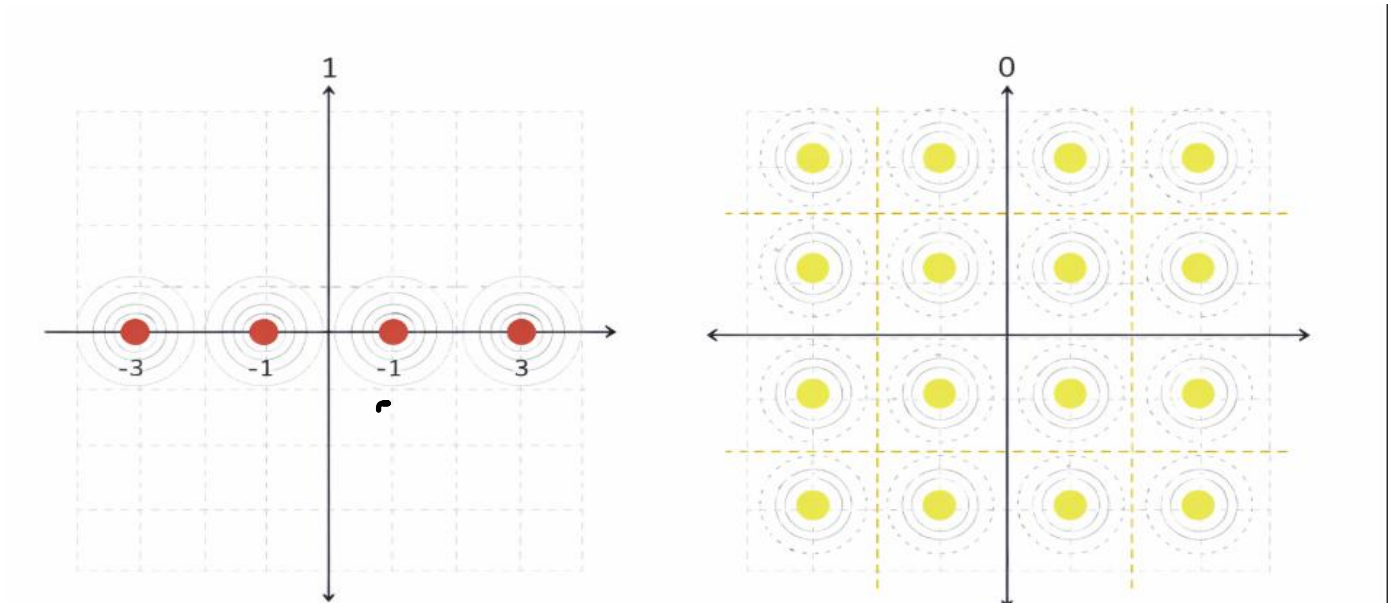



**Figure 9 analysis for 4ASK instead of 16QAM for simplicity**

- **Hand analysis**

$p(e/s_1) = p(e/s_4) = 0.5\,erfc\left(\sqrt{\frac{E_o}{N_0}}\right)$ **As S1 and S4 have one neighbor they have same probability of error**

$p(e/s_2) = p(e/s_3) = erfc\left(\sqrt{\frac{E_o}{N_0}}\right)$ **As S2 and S3 have two neighbors they have same probability of error**

$\therefore P_{error\ for\ 4ASK(\ two\ bit\ from\ 4\ of\ 16\ QAM)} = \frac{1}{4}\left(2p(e|s_1) + 2p(e|s_2)\right) = 0.75\,erfc\left(\sqrt{\frac{E_o}{N_0}}\right)$

**As the energy of each symbol not equal in ASK →** $Esav = \frac{2+9+9}{4}Eo = 5Eo$ **so the Eb =5Eo/2=2.5Eo then substitute to get the probability of error**

$\therefore P_{error\ for\ 4ASK} = 0.75\,erfc\left(\frac{\sqrt{E_b}}{2.5\,No}\right)$ **→** $P_{error}$ **of two bits of 16_QAM is** $0.75\,erfc\left(\frac{\sqrt{E_b}}{2.5\,No}\right)$

$P_{error} = 1-(1-P_{error\ of\ the\ 1^{st}\ 2bit})\ (1-P_{error\ of\ the\ 2^{nd}\ 2bit}) = 2*P_{error\ for\ 4ASK} - P_{error\ for\ 4ASK}{}^2$

$\therefore P_{error\ for\ 16QAM} = SER = 1.5erfc\left(\frac{\sqrt{E_b}}{2.5\,No}\right)$

$\therefore BER = \frac{SER}{4} = \frac{3}{8}erfc\left(\sqrt{\frac{E_b}{2.5 * N_0}}\right)$

## 5.4 Simulation Results

- Gray 16-QAM – Simulated vs. Theoretical BER

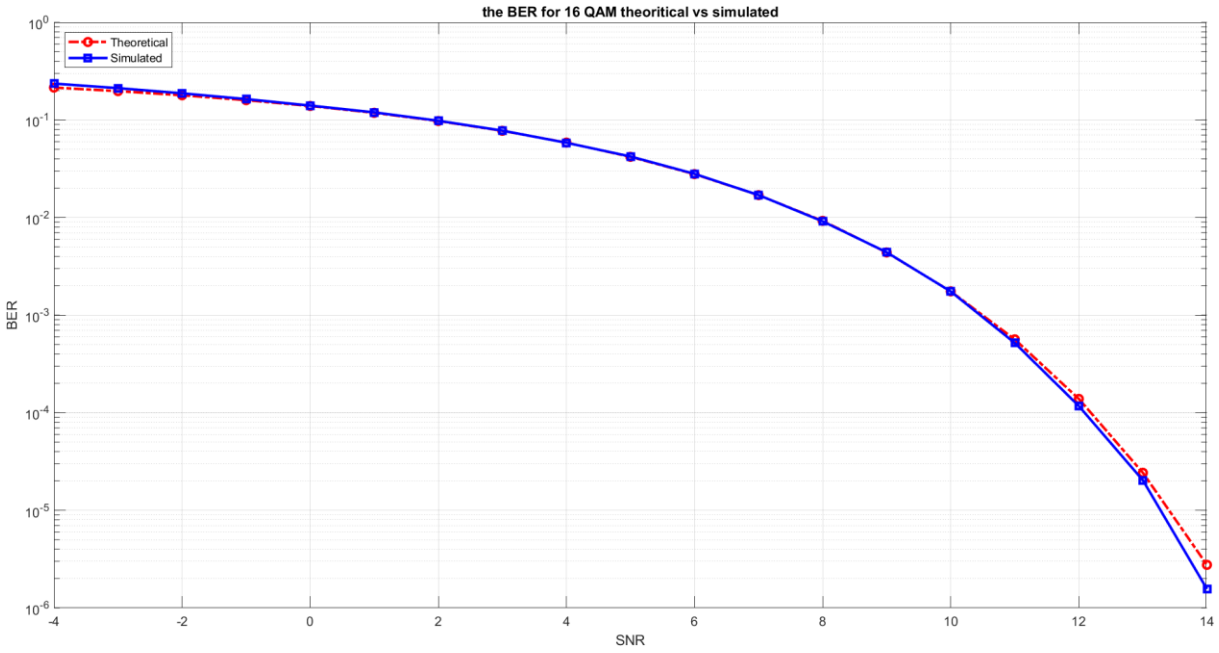



**Figure 10 Gray 16-QAM – Simulated vs. Theoretical BER**

### Comment:

The theoretical and simulated BER curves are very close across the entire SNR range. This confirms that the system has been correctly implemented, including the modulation, noise modeling, and demodulation processes. And the small deviations between the two curves, especially at lower SNR values, are mainly due to the finite number of transmitted symbols used in the simulation, as the theoretical results assume an infinite number of bits.

# 6. BFSK Analysis

## 6.1 BFSK System Overview

Binary Frequency Shift Keying (BFSK) is a digital modulation technique that transmits data by shifting the frequency of a carrier signal between two distinct values. It is a simple and robust modulation scheme, known for its good noise immunity and ease of implementation.

In BFSK, each symbol represents one bit, mapped to one of two different frequencies (typically $f_1$ and $f_2$). For example, a binary "0" may be transmitted using frequency $f_1$, while a binary "1" is transmitted using frequency $f_2$. Unlike BPSK, which varies the phase, BFSK varies the frequency of the carrier signal to encode information.

Although BFSK is less bandwidth-efficient than some higher-order modulation techniques, it performs well in noisy environments and is less sensitive to phase errors. This makes it suitable for applications where reliable data transmission is more important than spectral efficiency.

## 6.2 System Implementation

- **Mapper:**

In the BFSK transmitter, a random binary stream is generated, where each bit is treated as an independent symbol since each symbol carries one bit of information. Unlike higher-order modulation schemes, no bit grouping or decimal conversion is required.
Each input bit is mapped to one of two orthogonal frequency-based waveforms. Specifically, binary 0 is transmitted using a sinusoidal signal at frequency $f_1$, while binary 1 is transmitted using a sinusoidal signal at frequency $f_2$. These two frequencies represent the two distinct signaling states of the BFSK system.
The resulting transmitted signal is generated by selecting one of the two basis functions:

$$\phi_1(t) = \sqrt{\frac{2}{T_b}}\cos(2\pi f_1 t), \phi_2(t) = \sqrt{\frac{2}{T_b}}\cos(2\pi f_2 t)$$

For each bit in the input stream, the corresponding waveform is scaled by the symbol energy $\sqrt{E_b}$, producing the transmitted BFSK symbols. This mapping results in a signal set represented in a two-dimensional orthogonal signal space, where each bit is associated with a distinct frequency component rather than a change in amplitude or phase.

### Matlab Code:

```
%% Parameters
N = 1e5;            % Number of bits
Tb = 1;             % Bit duration
fs = 100;           % Sampling frequency
t = 0:1/fs:Tb-1/fs;   % Time vector

f1 = 1/Tb;          % Frequency for bit 0
f2 = 2/Tb;          % Frequency for bit 1

EbNo_dB = -4:1:14;    % Eb/No range in dB
BER = zeros(1, length(EbNo_dB));

Eb = 1;             % Energy per bit

%% Basis Functions (Orthonormal)
phi1 = sqrt(2/Tb) * cos(2*pi*f1*t);  % Basis function for bit 0
phi2 = sqrt(2/Tb) * cos(2*pi*f2*t);  % Basis function for bit 1

%% Generate Random Bits
bits = randi([0 1], N, 1);

%% MAPPER
% Map bits into BFSK waveforms
s = zeros(N, length(t));

for k = 1:N
   if bits(k) == 0
      s(k,:) = sqrt(Eb) * phi1;   % Bit 0 → frequency f1
   else
      s(k,:) = sqrt(Eb) * phi2;   % Bit 1 → frequency f2
   end
```

```
end

%% Simulation over Eb/No
for i = 1:length(EbNo_dB)

    EbNo = 10^(EbNo_dB(i)/10);   % Convert dB to linear
    N0 = Eb / EbNo;              % Noise spectral density
```

- **Channel:**

The transmitted BFSK signals are passed through an Additive White Gaussian Noise (AWGN) channel to model realistic communication conditions. A range of $E_b/N_0$ values from −4 dB to 14 dB is used to evaluate the system performance under different noise levels, ranging from low SNR (high noise) to high SNR (low noise).

For each $E_b/N_0$ value, it is first converted from dB to linear scale in order to compute the noise power spectral density $N_0$. The noise variance is then determined based on the bit energy $E_b$. Since BFSK is implemented using real-valued orthogonal basis functions in this simulation, real Gaussian noise is generated using randn and properly scaled according to the sampling and noise variance.

Specifically, because each bit is represented by a waveform sampled over multiple time instances, the noise is adjusted to match the signal energy distribution over the sampling interval. This ensures that the received signal maintains a correct signal-to-noise ratio consistent with the theoretical $E_b/N_0$ definition.

The generated noise is then added to the transmitted BFSK waveforms, producing the received noisy signals. These signals are subsequently processed by correlating with the two orthogonal basis functions to determine which frequency was transmitted, enabling bit detection at the receiver.

## Matlab Code:

```
%% CHANNEL
% Add white Gaussian noise to the transmitted signal
noise = sqrt(N0 * fs / 2) * randn(size(s));
r = s + noise;   % Received signal
```

- **Demapper:**

At the receiver, the noisy BFSK signals are detected using a **correlator-based decision rule** rather than a simple threshold. Since BFSK uses two orthogonal basis functions, each received signal is projected onto both basis functions to determine its similarity to each possible transmitted waveform.

Specifically, the received signal is correlated with:

$$\phi_1(t) = \sqrt{\frac{2}{T_b}}\cos(2\pi f_1 t), \phi_2(t) = \sqrt{\frac{2}{T_b}}\cos(2\pi f_2 t)$$

This is implemented by computing the inner products:

- $r_1 = \int r(t)\phi_1(t)\,dt$
- $r_2 = \int r(t)\phi_2(t)\,dt$

The decision rule is then based on selecting the frequency component with the higher correlation value. If $r_1 > r_2$, the receiver decides binary 0 (frequency $f_1$); otherwise, it decides binary 1 (frequency $f_2$).

The detected bits are then directly reconstructed from this comparison without any need for additional mapping or grouping.

The Bit Error Rate (BER) is computed by comparing the detected bit stream with the transmitted bit stream and dividing the number of bit errors by the total number of transmitted bits $N$. This process is repeated for all $E_b/N_0$values to evaluate the system performance under different noise conditions.

## Matlab Code:

```
%% DEMAPPER

    % Correlator: project received signal onto basis functions
    r1 = trapz(t, r .* phi1, 2);   % Projection on phi1
    r2 = trapz(t, r .* phi2, 2);   % Projection on phi2

    % Decision rule:
    % Choose the basis with the higher correlation
    rx_bits = r2 > r1;

    % Bit Error Rate calculation
    BER(i) = sum(bits ~= rx_bits) / N;
end

%% Theoretical BER (Coherent BFSK)
EbNo_linear = 10.^(EbNo_dB/10);
BER_theoretical = 0.5 * erfc(sqrt(EbNo_linear/2));
```

## 6.3 Simulation Results

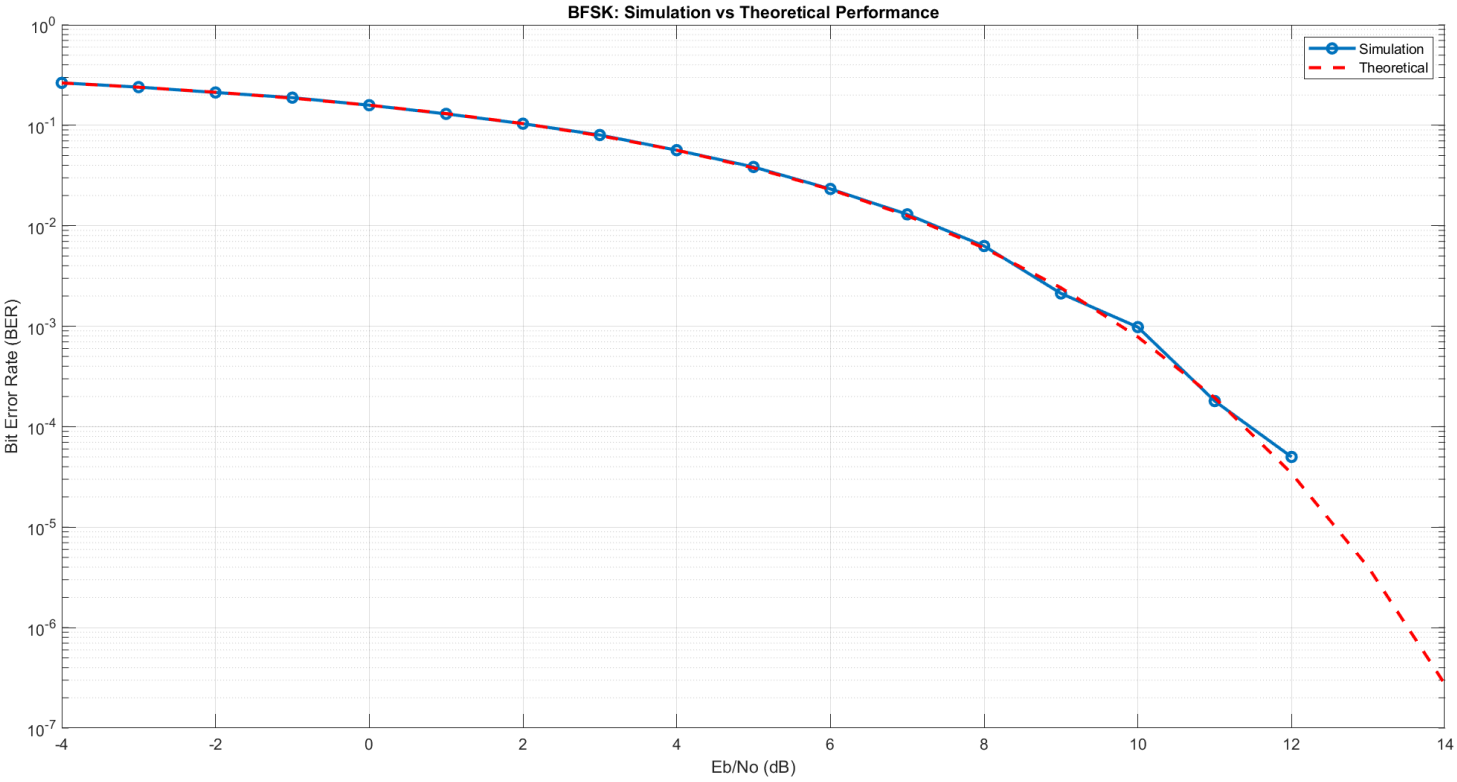



**Figure 11 BFSK – Simulated vs. Theoretical BER**

## Comment:

The obtained results demonstrate a very close agreement between the simulated BER and the theoretical BER for the BFSK system, which indicates that the simulation has been correctly implemented and accurately reflects the expected analytical performance. Across most values of $E_b/N_0$, the simulated curve follows the theoretical trend with high precision, especially in the low to moderate SNR region.

At higher $E_b/N_0$values, a slight divergence between the two curves can be observed. This discrepancy is primarily due to the limited number of transmitted bits in the simulation. As the noise level decreases, the probability of bit errors becomes extremely small, making it unlikely to capture sufficient error events within a finite simulation length. Consequently, the simulated BER may reach very low values or appear to flatten.

Additionally, since the BER is plotted on a logarithmic scale, extremely small or zero error counts cannot be properly represented (as log (0)is undefined). This may cause the simulated curve to appear less smooth or to deviate slightly from the theoretical curve. On the other hand, the theoretical BER, derived from analytical expressions, continues to decrease smoothly without such limitations.

## 6.3 Theoretical Analysis

- **Basis functions:**

The BFSK signal set is represented using two orthonormal basis functions corresponding to two different carrier frequencies $f_1$ and $f_2$. These basis functions are defined as:

$$\phi_1(t) = \sqrt{\frac{2}{T_b}} \cos(2\pi f_1 t)$$

$$\phi_2(t) = \sqrt{\frac{2}{T_b}} \cos(2\pi f_2 t)$$

where $T_b$ is the bit duration.

These functions are normalized to have unit energy and are orthogonal to each other due to the difference in frequencies $f_1 \neq f_2$. Therefore, they satisfy:

$$\int_0^{T_b} \phi_1(t)\phi_2(t)\, dt = 0$$

Hence, the BFSK signal space is two-dimensional, where each transmitted bit is represented as a linear combination of these orthonormal basis functions.

- **Baseband Equivalent Signal Representation:**

The BFSK passband signals can be expressed in their baseband equivalent form using orthonormal basis functions. The transmitted signal depends on the transmitted bit $b \in \{0,1\}$.
Let the carrier frequencies be $f_1$ and $f_2$, and the bit duration be $T_b$. The passband BFSK signals are:
For bit 0:

$$s_0(t) = \sqrt{E_b}\, \phi_1(t) \cos(2\pi f_1 t)$$

For bit 1:

$$s_1(t) = \sqrt{E_b}\, \phi_2(t) \cos(2\pi f_2 t)$$

Using the orthonormal basis representation, the baseband equivalent signal vector form is:

$$s(t) = \begin{cases} \sqrt{E_b}\,[1 \quad 0], & \text{for bit 0} \\ \sqrt{E_b}\,[0 \quad 1], & \text{for bit 1} \end{cases}$$

- **Carrier Frequencies:**

The modulation uses two orthogonal carrier frequencies:

$$f_1 = \frac{1}{T_b}, f_2 = \frac{2}{T_b}$$

- **Interpretation:**

This means that each bit is represented as a point in a two-dimensional signal space, where:

- bit 0 lies along the $\phi_1(t)$axis
- bit 1 lies along the $\phi_2(t)$axis

## 6.4 Simulated PSD

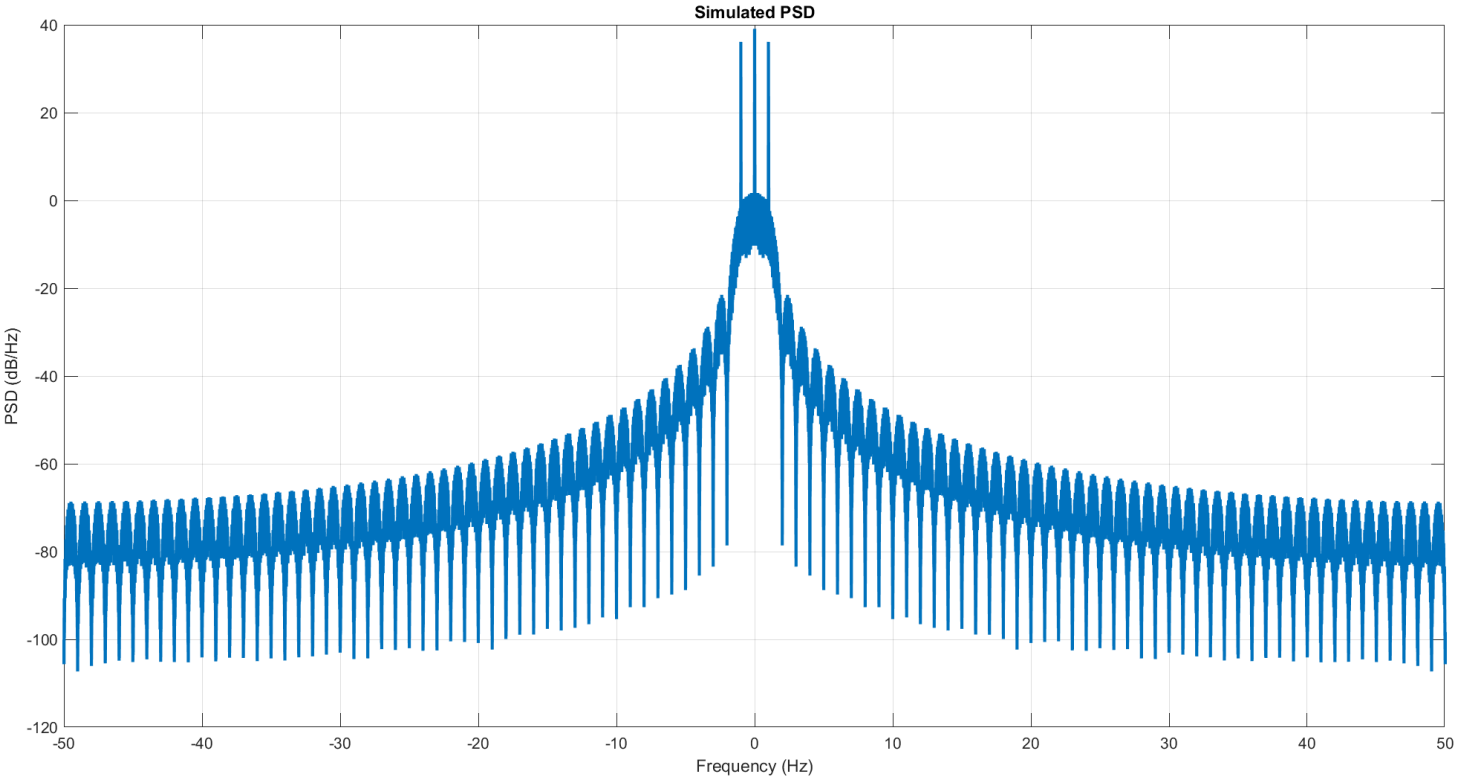



**Figure 12 BFSK – Simulated PSD**

**Comment:**

The simulated PSD of the BFSK baseband signal exhibits a main lobe centered around zero frequency, with noticeable spectral components resulting from the frequency shift $\Delta f$ between the two symbols. The presence of ripples and sidelobes is due to the finite bit duration $T_b$ and the rectangular pulse shaping used in the simulation. Additionally, the spectrum shows symmetry around zero, as expected for a real-valued baseband signal. The overall bandwidth is primarily determined by $\Delta f$ and the symbol rate, confirming the theoretical expectation that increasing frequency separation leads to a wider spectrum.

# 7. BER vs. Eb/No Comparison

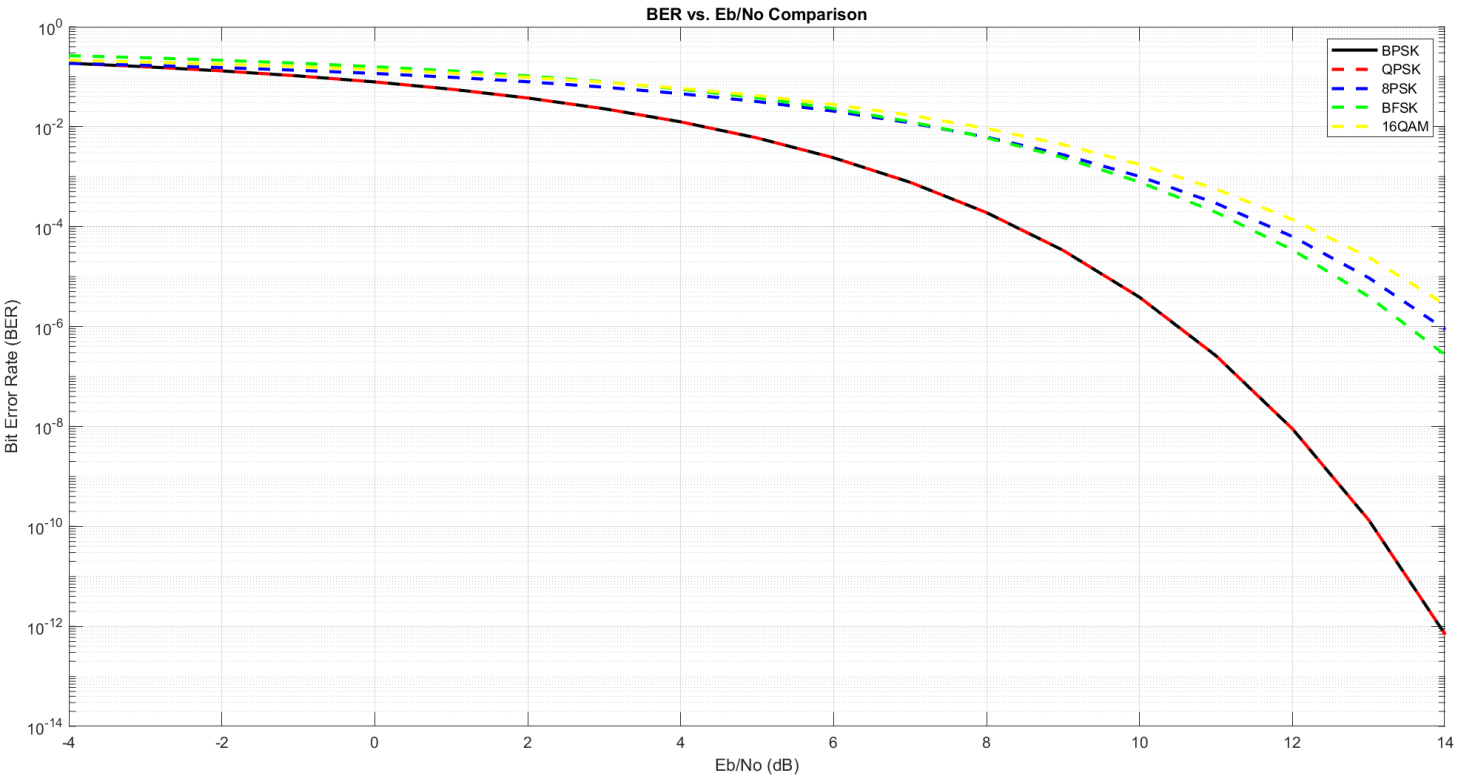



**Figure 13 BER – Comparison**

## Comment:

The plotted BER curves match the expected theoretical behavior of digital modulation schemes over an AWGN channel. As the $E_b/N_0$increases, all modulation schemes show a decreasing BER, which reflects improved signal reliability as the effect of noise becomes less significant.

BPSK and QPSK provide the best performance among all schemes, and their curves almost completely overlap. This is expected because both have the same minimum distance between constellation points, which is the key factor determining error performance. In QPSK, although more bits are transmitted per symbol, the energy per bit remains effectively the same as in BPSK, leading to identical BER performance.

8-PSK shows worse performance compared to BPSK and QPSK. This is because increasing the number of phase states reduces the angular separation between symbols, making them more susceptible to noise and increasing the probability of incorrect detection. As a result, a higher $E_b/N_0$is required to achieve the same BER.

BFSK exhibits intermediate performance. Its behavior depends on the detection method, but in general, it has a smaller effective distance between symbols compared to BPSK, which leads to a slightly higher BER under the same conditions.

16-QAM has the worst performance among the plotted schemes. This is due to its high constellation density, where many symbols are packed closely together in both amplitude and phase. This small spacing makes the system highly sensitive to noise, resulting in a higher probability of symbol errors, especially at lower $E_b/N_0$ values.

Overall, the results clearly illustrate the fundamental trade-off in digital communications: lower-order modulation schemes (like BPSK and QPSK) provide better error performance and robustness to noise, while higher-order schemes (like 16-QAM) achieve higher data rates at the expense of increased BER.

# 8. Conclusion

This project presented a comprehensive analysis and simulation of several digital modulation schemes, including BPSK, QPSK, 8PSK, BFSK, and 16-QAM over an AWGN channel. The performance of each scheme was evaluated in terms of Bit Error Rate (BER) as a function of Eb/No and compared with the corresponding theoretical results.

The simulation results showed a strong agreement with the theoretical analysis across all modulation schemes, which validates the correctness of the implemented models and detection techniques. Minor deviations at high Eb/No values were observed due to the finite number of transmitted bits in the simulation, leading to limited error occurrences.

From a performance perspective, BPSK and QPSK demonstrated the best error performance due to their larger minimum distance between constellation points, making them more robust against noise. Gray-coded QPSK further proved to be more efficient than non-Gray mapping by minimizing bit errors per symbol error. On the other hand, higher-order modulation schemes such as 8PSK and 16-QAM achieved better spectral efficiency by transmitting more bits per symbol, but at the cost of increased sensitivity to noise and higher BER.

BFSK showed moderate performance, benefiting from its orthogonality and robustness, but still requiring more bandwidth compared to phase modulation schemes. Overall, the results clearly highlight the fundamental trade-off between spectral efficiency and noise immunity in digital communication systems.
In conclusion, the choice of modulation scheme depends on system requirements: lower-order schemes are preferred in noisy environments for reliable communication, while higher-order schemes are suitable when bandwidth efficiency is a priority.

# 9. Appendix

https://github.com/FatmaFoley/Digital_Communicatin_Project3